CSE 598 008
EECS 498 003
Lab 8

Tianyiming (Henry) Jing

October 23, 2026

Based on slides by Ivan Wei (Fall 2025).

Reminder

- Problem Set 3 (Chapter 5) due October 26, 11:59pm
- Start looking for Project 1 partners (released after PS3)

Today

- Midterm solution review
- Distributed lock exercise, continued from Lab 7
  - We will go over the answer today
- Which first: the midterm review or the exercise? Your call
- Questions about PS3? Ask anytime

Distributed System Model

```
datatype MessageOps = MessageOps(
    recv: Option<Message>,
    send: Option<Message>)

datatype Step =
    | HostActionStep(hostid: HostId,
                     msgOps: MessageOps)

ghost predicate NextStep(c, v, v', step)
{
    && HostAction(c, v, v', step.hostid,
                  step.msgOps)
    && Network.Next(c.network, v.network,
                    v'.network, step.msgOps)
}
```

- A distributed system is a state machine made of hosts and a network
- In each step, one host takes a step together with the network; every other host stays the same
- `msgOps` is a binding variable: the host and the network must agree on what is received and what is sent
- A host can only receive a message the network has; hosts never read each other's state

Network Models

- Lecture and PS3: the network keeps every message ever sent (`sentMsgs`)
  - A message can be received over and over, in any order
  - This is also how several hosts can hear the same message
- This exercise: a message can be received at most once, in any order
  - You design the network's `Variables`, `Init`, and `Next`
- Think about it: what could go wrong for the lock if a message could be received twice?
- The network model is trusted, not proven: if it is wrong, the proof says nothing about the real system

Proof Exercise

- Continuing from Lab 7: Distributed Lock with Network
- Starter code: `lab08.dfy` on the course GitHub
  - `UtilitiesLibrary.dfy` is in the same folder
- We will go over the answer today

Extra - All Horses Are the Same Color

- Claim: in any group of n horses, all horses have the same color
- Base case (n = 1): a single horse has the same color as itself
- Inductive step: take any group of n + 1 horses
  - Leave out the first horse: the other n have the same color
  - Leave out the last horse: the other n have the same color
  - The horses in the middle are in both groups, so all n + 1 have the same color
- So all horses are the same color!

Extra - All Horses Are the Same Color

- The inductive step fails going from 1 horse to 2
  - Leave out the last horse: only horse 1 is left. Leave out the first: only horse 2
  - No horse is in both groups, so nothing links their colors
- Every other step works, but one broken step breaks the whole proof
- Dafny code: `all_horses_same_color.dfy` in the extra folder on the course GitHub



https://en.wikipedia.org/wiki/All_horses_are_the_same_color